Analisis sobre autoencoders y clasificadores convolucionales

Victoria Lopez*
Facultad de Matemática, Astronomía, Física y Computación,
Universidad Nacional de Córdoba, Córdoba, Argentina

(Dated: February 23, 2026)

En este trabajo se implementa un autoencoder convolucional y un clasificador sobre el dataset Fashion-MNIST para evaluar el impacto del pre-entrenamiento. El autoencoder se entrenó con MSE y luego su encoder se reutilizó en la etapa de clasificación. Se compararon cuatro esquemas: pre-entrenado congelado, desde cero, fine-tuning y encoder aleatorio congelado.

1. INTRODUCCIÓN

En el aprendizaje automático, los autoencoders son modelos de aprendizaje no supervisado que permiten descubrir estructuras latentes en los datos mediante la compresión y reconstrucción de la información. Están compuestos por un encoder, que reduce la dimensión de la entrada a un espacio latente, y un decoder, que reconstruye los datos originales a partir de dicha representación.

En el caso de imágenes, las redes neuronales convolucionales son especialmente adecuadas por su capacidad para capturar patrones espaciales. En este trabajo se implementa un autoencoder convolucional en PyTorch sobre el dataset Fashion-MNIST (imágenes en escala de grises de 28×28 píxeles), con el objetivo de analizar el espacio latente y evaluar la calidad de las reconstrucciones obtenidas.

2. FUNDAMENTOS TEÓRICOS

2.1. Redes Convolucionales y Autoencoders

Las Redes Neuronales Convolucionales (CNN) son modelos diseñados para procesar datos con estructura espacial, como imágenes, mediante la aplicación de filtros convolucionales que extraen automáticamente patrones locales relevantes (Goodfellow et al. 2016, LeCun et al. 1998). Cada capa de convolución genera mapas de características cuya dimensión depende de parámetros como el tamaño del kernel, el *stride* y el *padding*. Estas capas suelen combinarse con funciones de activación no lineales (por ejemplo, ReLU) y operaciones de *pooling*, que reducen la dimensión espacial preservando la información más significativa y mejorando la eficiencia computacional (LeCun et al. 1998). Finalmente, las representaciones extraídas pueden conectarse a capas densas para tareas de clasificación, como en arquitecturas profundas de visión por computadora (Krizhevsky et al. 2012).

Los **autoencoders** son modelos de aprendizaje no supervisado que aprenden representaciones comprimidas de los datos mediante un esquema de codificación y reconstrucción (Goodfellow et al. 2016, Hinton and Salakhutdinov 2006). Están compuestos por un *encoder*, que transforma la entrada x en una representación latente z, y un *decoder*, que reconstruye una aproximación $\hat{x}$. El entrenamiento se realiza minimizando una función de pérdida de reconstrucción, típicamente el error cuadrático medio (MSE), que mide la discrepancia entre la entrada original y su reconstrucción.

En el caso de imágenes, los autoencoders pueden implementarse utilizando capas convolucionales y convoluciones transpuestas, permitiendo preservar la estructura espacial de los datos. El espacio latente actúa como un cuello de botella que obliga al modelo a capturar las características más relevantes, lo que resulta útil para reducción de dimensionalidad, eliminación de ruido y aprendizaje de representaciones reutilizables.

3. RESULTADOS

El conjunto de entrenamiento original de FashionMNIST (60.000 imágenes) fue dividido en subconjuntos de entrenamiento (80%) y validación (20%). El autoencoder fue entrenado utilizando exclusivamente este conjunto, sin emplear etiquetas. Posteriormente, el clasificador supervisado fue entrenado utilizando la misma partición, manteniendo congelados los parámetros del codificador previamente entrenado. El conjunto de test (10.000 imágenes) fue reservado exclusivamente para la evaluación final del modelo

3.1. Análisis de resultados: entrenamiento y selección de hiperparámetros

TABLE I: Arquitectura del módulo codificador

Capa	Tipo	Entrada	Salida	Kernel	Stride	Padding
1	Conv2D	$1 \times 28 \times 28$	$32 \times 28 \times 28$	3×3	1	1
2	MaxPool2D	$32 \times 28 \times 28$	$32 \times 14 \times 14$	2×2	2	0
3	Conv2D	$32 \times 14 \times 14$	$64 \times 14 \times 14$	3×3	1	1
4	MaxPool2D	$64 \times 14 \times 14$	$64 \times 7 \times 7$	2×2	2	0
5	Conv2D	$64 \times 7 \times 7$	$128 \times 7 \times 7$	3×3	1	1

La dimensión del espacio latente es $128 \times 7 \times 7$.

TABLE II: Arquitectura del módulo decodificador

Capa	Tipo	Entrada	Salida	Kernel	Stride	Padding
1	ConvTranspose2D	$128 \times 7 \times 7$	$64 \times 14 \times 14$	2×2	2	0
2	ConvTranspose2D	$64 \times 14 \times 14$	$32 \times 28 \times 28$	2×2	2	0
3	Conv2D	$32 \times 28 \times 28$	$1 \times 28 \times 28$	3×3	1	1

Entrenamos un autoencoder convolucional sobre el conjunto de entrenamiento con la idea de obtener un encoder que luego pudiera reutilizarse para clasificación. La arquitectura utilizada se muestra en las Tablas I y II, donde puede verse cómo la imagen se va comprimiendo progresivamente hasta llegar al espacio latente $128 \times 7 \times 7$.

Durante el entrenamiento probamos distintas configuraciones, principalmente variando la cantidad de filtros y la tasa de aprendizaje. Cuando el modelo tenía menos filtros, las reconstrucciones perdían definición y las prendas se veían más borrosas. En cambio, arquitecturas más grandes no mejoraban demasiado el resultado visual y hacían el entrenamiento más pesado. Por eso, la configuración final (32, 64 y 128 filtros), detallada en la Tabla I, resultó ser la más equilibrada.

TABLE III: Hiperparámetros principales del autoencoder

Parámetro	**Valor**
Dataset	Fashion-MNIST
Función de pérdida	MSE
Optimizador	Adam
Tasa de aprendizaje	1×10^{-3}
Batch size	128
Épocas	20
Filtros (encoder)	32, 64, 128
Dimensión latente	$128 \times 7 \times 7$

En cuanto a los hiperparámetros de entrenamiento (Tabla III), ajustamos la tasa de aprendizaje hasta encontrar un valor estable. Con 10^{-3} y el optimizador Adam el modelo aprendía de forma consistente y sin comportamientos inestables. El tamaño de batch de 128 también ayudó a mantener un entrenamiento equilibrado.

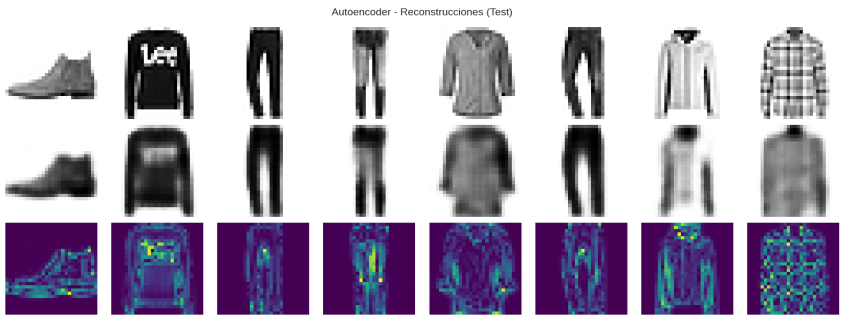


FIG. 1: Imágenes originales (fila superior) y reconstrucciones generadas por el autoencoder.

La Figura 3 muestra ejemplos de imágenes originales y sus reconstrucciones en el conjunto de prueba. Se observa que el modelo logra conservar bien la forma general de las prendas, manteniendo contornos y proporciones. Aunque algunos detalles finos se suavizan, la estructura principal se mantiene. Esto indica que el espacio latente aprendido captura la información más importante del dataset sin simplemente copiar las imágenes.

En conjunto, los resultados mostrados en las Tablas I–III y en la Figura 3 muestran que la arquitectura y los hiperparámetros seleccionados permiten una buena reconstrucción y generan una representación útil para la etapa posterior de clasificación.

3.2. Clasificador con Encoder Pre-entrenado

TABLE IV: Arquitectura del cabezal clasificador.

Capa	**Entrada**	**Salida**	**Kernel**	**Stride**	**Padding**
Conv, ReLU, Dropout	$128 \times 7 \times 7$	$32 \times 7 \times 7$	3×3	1	1
AvgPool	$32 \times 7 \times 7$	$32 \times 1 \times 1$	7×7	1	0
Flatten	$32 \times 1 \times 1$	32	–	–	–
Linear, ReLU, Dropout	32	128	–	–	–
Linear, LogSoftmax	128	10	–	–	–

El cabezal clasificador recibe la representación latente del encoder ($128 \times 7 \times 7$) y aplica una convolución seguida de *Adaptive Average Pooling* para reducir la dimensión espacial a 1×1. Luego, dos capas totalmente conectadas ($32 \rightarrow 128 \rightarrow 10$) con activaciones ReLU y *dropout* ($p = 0.3$) realizan la clasificación. La salida utiliza *LogSoftmax* para obtener las probabilidades de las 10 clases.

El modelo completo cuenta con 47,274 parámetros entrenables cuando el encoder no está congelado.

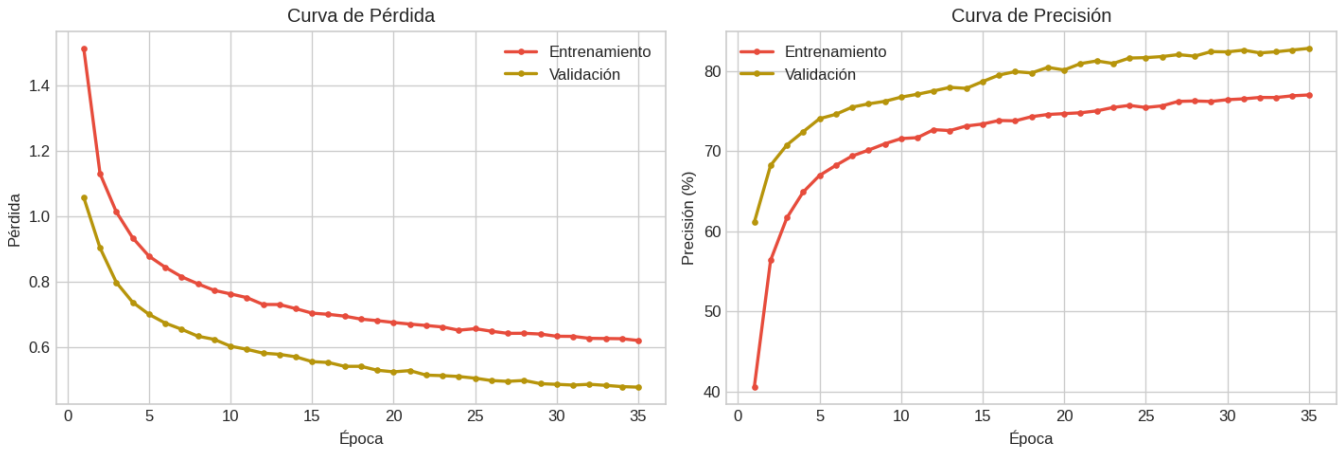


FIG. 2: Curvas de Perdida y Precision

En las curvas se muestra la evolución de la pérdida y la precisión para el esquema Original, evaluadas sobre los conjuntos de entrenamiento (rojo) y validación (amarillo oscuro). La pérdida disminuye de forma sostenida y la precisión aumenta progresivamente en ambos conjuntos, sin divergencias marcadas entre ellos. Esto indica un entrenamiento estable y una adecuada capacidad de generalización.

TABLE V: Precisión por clase – Esquema Original (encoder pre-entrenado congelado)

Clase	**Precisión (%)**
Camiseta (T-shirt/top)	86.1
Pantalón (Trouser)	93.3
Pullover	63.7
Vestido (Dress)	83.7
Abrigo (Coat)	81.0
Sandalia (Sandal)	92.2
Camisa (Shirt)	35.9
Zapatilla (Sneaker)	92.8
Bolso (Bag)	95.7
Botín (Ankle boot)	95.3
Accuracy Global (Test)	**81.97**

Se observa que las clases correspondientes a calzado y accesorios (Sandalia, Zapatilla, Bolso y Botín) presentan las mayores precisiones, superando el 92%. En contraste, la clase Camisa (Shirt) obtiene el valor más bajo (35.9%), lo que sugiere una mayor dificultad para distinguirla de otras prendas superiores como Camiseta, Pullover o Abrigo. Esta diferencia puede explicarse por la similitud visual entre estas categorías, que comparten forma y textura, lo que incrementa la ambigüedad en la clasificación.

3.3. Comparación de Estrategias de Entrenamiento

La Tabla VI presenta la comparación entre los cuatro esquemas de entrenamiento evaluados.

El esquema **Original (pre-entrenado congelado)** utiliza el encoder previamente entrenado con el autoencoder y mantiene sus pesos fijos durante la clasificación. Solo se entrena el cabezal clasificador. Este método alcanzó una precisión en test del 81.97%, mostrando un desempeño razonable pero limitado por la falta de ajuste fino del encoder.

El esquema **Desde Cero** entrena toda la red desde inicialización aleatoria, sin utilizar pre-entrenamiento. Este modelo obtuvo la mayor precisión en test (88.30%), lo que indica que, en este caso, el entrenamiento completo permitió aprender representaciones más adecuadas para la tarea de clasificación.

El esquema **Fine-tuning** parte de un encoder pre-entrenado, pero permite actualizar sus pesos con una tasa de aprendizaje reducida durante la clasificación. Alcanzó una precisión en test del 84.88%, mejorando respecto al encoder congelado, aunque sin superar al entrenamiento desde cero.

Finalmente, el esquema **Random Frozen** utiliza un encoder inicializado aleatoriamente cuyos pesos no se actualizan. Este enfoque funcionó como línea base y obtuvo el peor desempeño (73.43%), confirmando que un encoder sin entrenamiento previo ni ajuste no proporciona representaciones útiles.

En conclusión, según los resultados obtenidos en el conjunto de test, el mejor desempeño final correspondió al esquema **Desde Cero**, con una precisión del 88.30%.

TABLE VI: Comparación final de los esquemas de entrenamiento

Esquema	**Val Acc (%)**	**Test Acc (%)**	**Mejor Época**	**Train Final (%)**
Original (Pre-entrenado congelado)	82.81	81.97	35	77.00
Desde Cero	88.51	88.30	35	85.65
Fine-tuning	85.43	84.88	32	81.96
Random Frozen	73.84	73.43	34	61.71

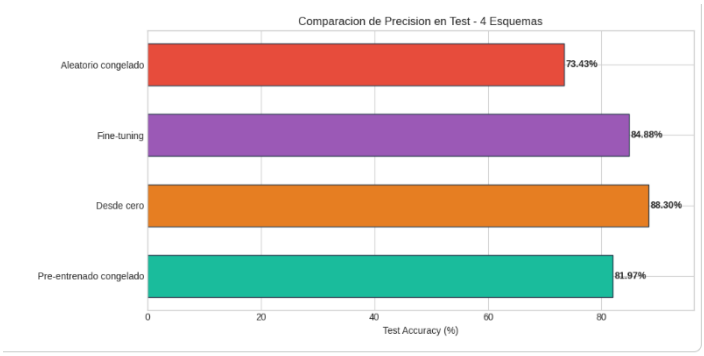
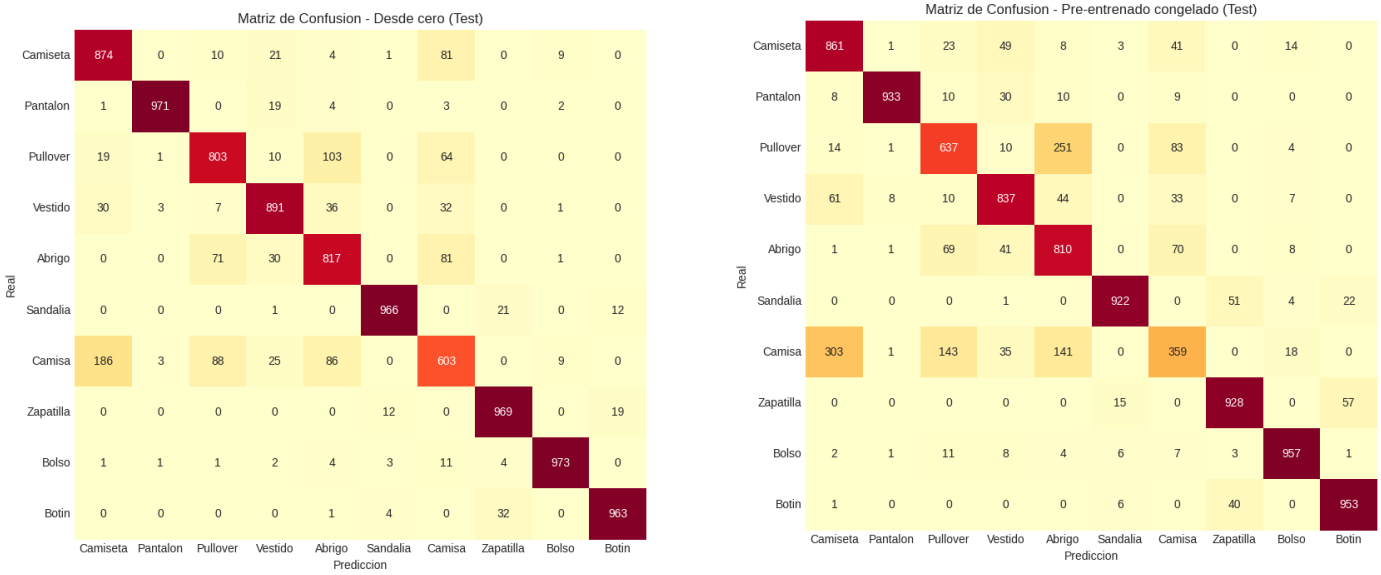


FIG. 3: Comparación de la precisión en el conjunto de test para los cuatro esquemas de entrenamiento evaluados.



(a) Entrenamiento desde cero

(b) Encoder pre-entrenado congelado

FIG. 4: Matrices de confusión normalizadas comparando el esquema con mejor rendimiento contra el que utiliza el encoder pre-entrenado.

4. DISCUSIÓN

La comparación entre los cuatro esquemas permite evaluar el impacto del pre-entrenamiento no supervisado. El hecho de que el *encoder* pre-entrenado congelado supere al *baseline* aleatorio (81.97% vs. 73.43%) indica que el autoencoder aprende representaciones latentes reutilizables para clasificación.

No obstante, el mejor desempeño se obtuvo entrenando el modelo desde cero (88.30%), lo que sugiere que, en un contexto con abundantes datos etiquetados como Fashion-MNIST (Xiao et al. 2017), la optimización *end-to-end* permite especializar todas las capas directamente en la tarea discriminativa. En contraste, el pre-entrenamiento con reconstrucción tiende a preservar información general de la imagen que no siempre resulta óptima para clasificación.

El *fine-tuning* mostró un rendimiento intermedio (84.88%), consistente con la idea de que el pre-entrenamiento puede aportar un buen punto de partida, aunque su ventaja disminuye cuando el régimen supervisado es amplio (Erhan et al. 2010). Las confusiones persistentes en la clase *Camisa* sugieren además una limitación asociada a la similitud visual entre categorías y a la baja resolución del dataset.

5. CONCLUSIONES

Se evaluó el impacto del pre-entrenamiento con autoencoders convolucionales en Fashion-MNIST (Xiao et al. 2017), comparando cuatro estrategias de entrenamiento. El pre-entrenamiento demostró aprender representaciones útiles, evidenciado por la mejora frente al *encoder* aleatorio congelado. Sin embargo, el entrenamiento supervisado completo desde cero alcanzó el mejor rendimiento.

En síntesis, el beneficio del pre-entrenamiento depende del contexto: resulta relevante cuando se parte de representaciones no informadas o en regímenes de datos limitados, pero no necesariamente supera al aprendizaje supervisado *end-to-end* cuando se dispone de abundantes datos etiquetados (Erhan et al. 2010).

*Victorialopez23@mi.unc.edu.ar

Erhan, D., Courville, A., Bengio, Y., and Vincent, P. (2010). Why does unsupervised pre-training help deep learning? *Journal of Machine Learning Research*, 11:625–660.

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.

Hinton, G. E. and Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507.

Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25.

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324.

Xiao, H., Rasul, K., and Vollgraf, R. (2017). Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*.